Lecture Notes: Loops & Flow Control

Nishut Suman 19 Nov, 2025 At 7:00 PM Notes Foundation Recommended Resource

Lecture Notes: Loops & Flow Control

Associated Lectures (1)

Description Discussions

Lecture Note: LOOPS IN PYTHON

Prerequisites:

- Understanding of Python variables and data types (strings, integers, lists, tuples)
- Familiarity with conditional statements (if, else, elif) **Time to Complete:** ~35–45 minutes **What You'll Be Able to Do:**
- Explain what loops are and when to use them
- Apply for and while loops to automate repetitive tasks
- Use break, continue, and pass statements effectively
- Work confidently with Python's range() function

1. Introduction: What Are Loops and Why Should You Care?

Core Definition

A **loop** in Python is an instruction that repeats a block of code multiple times as long as a condition is met. Loops make code **shorter, faster, and easier to manage** by automating repetitive actions.

A Simple Analogy

Think of a loop like setting an alarm clock to repeat daily. Once set, you don't need to reset it each day — it just runs until you stop it. **Limitation:** Unlike an alarm, a Python loop can also stop early when a specific condition changes.

Why This Matters to You

- **Efficiency:** Automate repetitive tasks instead of rewriting similar code.
- **Scalability:** Handle dynamic data (lists, files, inputs) with a few lines.
- **Maintainability:** Keep programs clean and easier to modify. **Real-world context:** Loops power automation in everything from processing web data to controlling iterations in AI training cycles.

2. The Foundation: Core Concepts Explained

Python provides two primary types of loops:

Type	Description	Common Use Case
for loop	Iterates over a sequence (list, tuple, string, etc.)	Processing items in a collection
while loop	Repeats code while a condition is True	Running until a condition changes

3. For Loops in Python

Definition

A **for loop** in Python is used to iterate through sequences like lists, tuples, strings, and ranges without manually managing the index.

Syntax

```
for var in iterable:
    # statements
    pass
```

Example 1: Basic For Loop

```
s = [""Geeks"", ""for"", ""Geeks""]
for i in s:
    print(i)
```

Output:

```
Geeks
for
Geeks
```

Example 2: Iterating Over a List

```
names = [""Ram"", ""Shyam"", ""Hari""]
for name in names:
    print(name)
```

Output:

```
Ram
Shyam
Hari
```

Example 3: Iterating Over Multiple Data Types

```
li = [""geeks"", ""for"", ""geeks""]
for x in li:
    print(x)
tup = (""geeks"", ""for"", ""geeks"")
for x in tup:
    print(x)
s = ""abc""
for x in s:
    print(x)
d = dict({'x':123, 'y':354})
for x in d:
    print(""%s  %d"" % (x, d[x]))
set1 = {10, 30, 20}
for x in set1:
    print(x)
```

Try it yourself: Run the code above and compare how iteration differs for lists, tuples, strings, dictionaries, and sets.

How It Works Internally

```
names = [""Ram"", ""Shyam"", ""Hari""]
iter_obj = iter(names)
while True:
    try:
        name = next(iter_obj)
        print(name)
    except StopIteration:
        break
```

This demonstrates how Python automatically uses iterators behind the scenes when executing a for loop.

4. While Loops in Python

Definition

A **while loop** executes a block of code repeatedly **as long as the condition evaluates to True**. When the condition becomes False, the loop stops.

Syntax

```
while condition:
    # code block
```

Example: Counting with While Loop

```
count = 1
while count <= 3:
    print(""Count is:"", count)
    count += 1
```

Output:

```
Count is: 1
Count is: 2
Count is: 3
```

Using else with While Loop

The **else block** executes only if the loop ends normally (not terminated by break).

```
count = 1
while count < 3:
    print(""Count:"", count)
    count += 1
else:
    print(""Loop finished"")
```

Output:

```
Count: 1
Count: 2
Loop finished
```

---

5. The range() Function

What It Does

range() returns a sequence of integers, starting from **0** (by default), incrementing by **1**, and stopping **before** the specified end value.

Syntax

```
range(start, stop, step)
```

Parameters

Parameter	Description	Default
start	Starting integer	0
stop	Ending integer (exclusive)	Required
step	Increment/decrement value	1

Example 1: Using Only stop

```
for i in range(5):
    print(i)
```

Example 2: Using start and stop

```
for i in range(5, 10):
    print(i)
```

Example 3: Using start, stop, and step

```
for i in range(1, 14, 3):
    print(i)
```

Example 4: Negative Step

```
for i in range(10, 0, -2):
    print(i)
```

Note: Using 0 as the step value raises a ValueError since the loop cannot progress. Official Reference: Python Docs — range() Function

6. Loop Control Statements

Control statements modify the normal flow of loops.

Statement	Purpose
break	Terminates the loop prematurely
continue	Skips the current iteration
pass	Does nothing; acts as a placeholder

Break Example

```
numbers = [1, 3, 5, 7, 8, 9, 10]
for num in numbers:
    if num % 2 == 0:
        print(""First even number found:"", num)
        break
print(""Loop terminated."")
```

Continue Example

```
numbers = [1, 2, 3, 4, 5]
print(""Numbers excluding even numbers:"")
for num in numbers:
    if num % 2 == 0:
        continue
    print(num)
```

Pass Example

```
def placeholder_function():
    pass
class EmptyClass:
    pass
x = 10
if x > 5:
    pass
else:
    print(""x is not greater than 5"")
```

7. Common Pitfalls and Recovery

Pitfall	Why It Happens	Recovery Tip
Infinite loop	Condition never becomes False	Ensure loop variables update properly
Misusing range()	Forgetting it's non-inclusive	Remember it stops before the end value
Using break inside nested loops incorrectly	Break only exits one loop level	Use flags or functions to manage multi-level exits

8. Check Your Understanding

What's the difference between a for loop and a while loop?

What happens if the while loop's condition never becomes False?

How would you use range() to print even numbers from 2 to 10?

What's the purpose of the pass statement?

Try predicting this code's output before running it:

```
for i in range(3):
    if i == 1:
        continue
    print(i)
```

9. Summary — Key Takeaways

- Use **for loops** to iterate over sequences directly.
- Use **while loops** for condition-controlled repetition.
- The **range()** function helps generate number sequences easily.
- Loop control statements (break, continue, pass) allow precise control.
- Always ensure loops terminate properly to avoid infinite cycles.

References

- Official Python Docs: Flow Control
- Python range() Function — Built-ins Reference